

Procedimientos almacenados

5. Procedimientos almacenados

Procedimientos almacenados

Hasta ahora, toda la lógica sobre tus datos vivía en tu aplicación Java: tanto con Spring Data JPA como con JDBC puro, era tu programa el que pedía las filas, las recorría y decidía qué hacer con ellas. En este apartado esa lógica cambia de sitio: se traslada al propio motor de base de datos, que la ejecuta él mismo, sin que tu aplicación tenga que traerse nada.

Qué es un procedimiento almacenado

Un procedimiento almacenado es código SQL (con lógica: variables, condiciones, bucles) que se guarda y se ejecuta dentro del propio gestor de base de datos, no en tu aplicación. Lo defines una vez, y a partir de ahí se invoca por su nombre.

Cada gestor tiene su propio lenguaje procedural — en PostgreSQL es PL/pgSQL, una extensión de SQL con variables, parámetros y control de flujo:

```
CREATE OR REPLACE PROCEDURE nombre_procedimiento(parametro TIPO)
LANGUAGE plpgsql
AS $$
BEGIN
    -- lógica con SQL normal aquí dentro
END;
$$;
```

Procedimiento vs. función

Ambos son código guardado en el gestor, pero una función siempre devuelve un valor y se puede usar dentro de una consulta (`SELECT mi_funcion(x)`); un procedimiento no tiene por qué devolver nada y se invoca de forma independiente con `CALL`. Cuando lo que quieres es "ejecutar una acción" más que "calcular un valor", un procedimiento encaja mejor.

Parámetros por posición

En parametro TIPO, entre paréntesis: cada parámetro se pasa por posición (el primer valor que envíes desde Java rellena el primer parámetro, el segundo el segundo, y así sucesivamente) y es de entrada (IN) por defecto — el procedimiento puede leerlo, pero no modificarlo y devolvértelo.

⚠ Si inviertes el orden al invocarlo

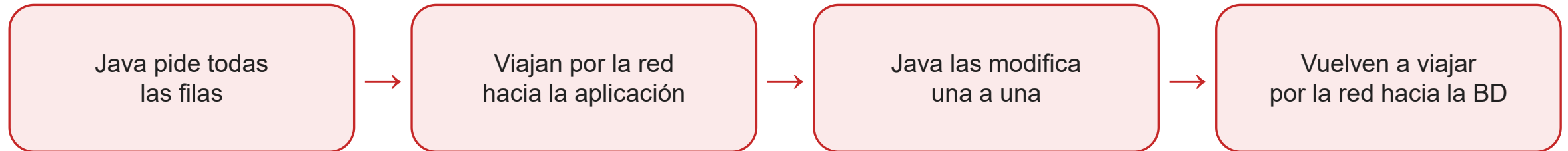
El motor no te avisa de nada raro: simplemente asigna cada valor al parámetro equivocado, y el error solo aparece cuando compruebas el resultado.

Cuándo tiene sentido usarlos — y cuándo no

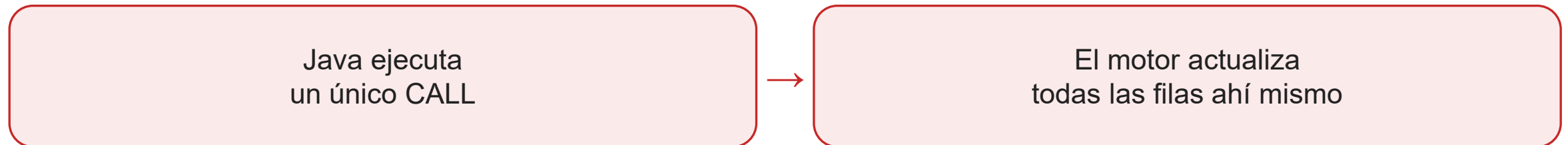
Tienen sentido cuando una operación conviene que ocurra atómicamente, cerca de los datos — por ejemplo, un ajuste masivo de precios que afecta a muchas filas a la vez: hacerlo dentro del motor evita traer todas las filas a Java, modificarlas una a una, y devolverlas, con todo el tráfico de red que eso implica.

Idas y vueltas por la red, con y sin procedimiento

Sin procedimiento almacenado



Con procedimiento almacenado



El inconveniente

La lógica queda repartida en dos sitios (tu código Java y la base de datos), lo que la hace más difícil de versionar junto con el resto del proyecto y de probar con las mismas herramientas que usas para el resto de tu código.

	Ventaja	Coste
Procedimiento almacenado	Una sola operación atómica, sin ida y vuelta por fila	Lógica repartida en dos sitios; más difícil de versionar y de probar
Bucle en Java	Todo el código en un solo sitio, fácil de testear	Un viaje de red por cada fila leída y modificada

Cómo se invocan desde Java

La vía estándar de JDBC es `CallableStatement` (el primo de `PreparedStatement`, pensado específicamente para invocar procedimientos) — se menciona para que sepas que existe, pero en este curso vas a usar una vía más cómoda: `JdbcTemplate`.

`JdbcTemplate` es el ayudante que trae Spring sobre JDBC puro: te evita exactamente la fontanería que has visto el apartado anterior (abrir conexión, cerrar recursos, gestionar excepciones a mano) sin llegar a ser un ORM completo como el que verás en detalle un poco más adelante.

Una sola línea

```
jdbcTemplate.update("CALL ajustar_precio_editorial(?, ?)", editorialId, porcentaje);
```

JdbcTemplate abre la conexión, prepara la sentencia con parámetros (igual de seguros frente a inyección SQL que el PreparedStatement que ya conoces), la ejecuta y cierra todo por ti.

Un ejemplo completo: ajustar_precio_editorial

Siguiendo con la aplicación de la librería: imagina que una editorial sube (o baja) el precio de todo su catálogo en un porcentaje. Es el caso perfecto para un procedimiento almacenado — una operación masiva, atómica, cerca de los datos.

1. El procedimiento en PostgreSQL

```
CREATE OR REPLACE PROCEDURE ajustar_precio_editorial(p_editorial_id BIGINT, p_porcentaje
NUMERIC)
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE libro
    SET precio = ROUND(precio * (1 + p_porcentaje / 100.0), 2)
    WHERE editorial_id = p_editorial_id;
END;
$$;
```

Sube o baja, de una sola vez, el precio de todos los libros de una editorial en el porcentaje indicado.

Por qué ROUND(..., 2) importa

precio es una columna con precisión 10,2 en la base de datos — sin redondear explícitamente a dos decimales, el cálculo ($\text{precio} * (1 + \text{porcentaje}/100)$) podría dejar más decimales de los que la columna admite con precisión limpia.

2. La invocación desde el service

```
@Service
@RequiredArgsConstructor
public class EditorialService {
    private final EditorialRepository editorialRepository;
    private final JdbcTemplate jdbcTemplate;

    @Transactional
    public void ajustarPrecio(Long editorialId, BigDecimal porcentaje) {
        if (!editorialRepository.existsById(editorialId)) {
            throw new ResponseStatusException(HttpStatus.NOT_FOUND, "Editorial no encontrada");
        }
        jdbcTemplate.update("CALL ajustar_precio_editorial(?, ?)", editorialId, porcentaje);
    }
}
```

Se inyecta como cualquier otro bean

JdbcTemplate se inyecta exactamente igual que cualquier otro bean — junto al EditorialRepository, con @RequiredArgsConstructor, sin ninguna configuración especial: Spring Boot lo configura automáticamente en cuanto detecta un DataSource en el classpath.

La comprobación de "editorial no encontrada" la sigues haciendo con Spring Data JPA (existsById) antes de invocar el procedimiento — no hace falta que todo pase por JdbcTemplate, solo la parte que de verdad conviene ejecutar cerca de los datos.

El @Transactional de arriba

Es el mismo que ya has visto en `update()/delete()` del apartado anterior: aquí también hay una lectura (`existsById`) seguida de una escritura (la llamada al procedimiento), así que el método entero se trata como una sola unidad todo-o-nada.

Todavía falta un paso

Con el procedimiento y el método del service ya tienes la lógica completa — pero, igual que con cualquier otra operación de tu API, todavía no es alcanzable desde fuera. Falta el último paso de siempre: un endpoint en el controller que llame a `ajustarPrecio(...)` y lo exponga por HTTP.

¿Por qué un procedimiento y no un bucle en Java?

También podrías resolver esta operación desde Java:

- Recuperar todos los libros de una editorial.
- Recorrerlos con un bucle.
- Actualizar su precio uno por uno.

El procedimiento almacenado sigue otro enfoque: la lógica ya está guardada dentro de PostgreSQL y la aplicación simplemente la invoca mediante CALL. Así, el gestor actualiza todos los libros sin tener que enviarlos primero a Java ni recorrerlos individualmente.

La idea importante

En un procedimiento almacenado, la aplicación solicita una operación, pero la lógica que la ejecuta está almacenada dentro de la propia base de datos.

En este apartado, el objetivo es aprender a...

- Crear un procedimiento almacenado.
- Pasarle parámetros.
- Invocarlo desde Java mediante JdbcTemplate.

Antes de cerrar el tema: consultas derivadas por nombre

En "Operaciones CRUD y gestión de transacciones" viste que puedes declarar una consulta propia sin escribir SQL, con un método sin cuerpo — como `findByEditorialId`. Ahí quedó como una idea suelta; aquí ves la convención completa.

Spring Data JPA lee el nombre del método pieza a pieza y lo traduce a una consulta. Estos son los patrones más habituales, todos sobre `LibroRepository`:

Los patrones más habituales

Método	Qué genera
<code>findByTitulo(String titulo)</code>	<code>WHERE titulo = ?</code>
<code>findByPrecioLessThan(BigDecimal precio)</code>	<code>WHERE precio < ?</code>
<code>findByTituloContainingIgnoreCase(String fragmento)</code>	<code>WHERE LOWER(titulo) LIKE LOWER('%fragmento%')</code>
<code>findByEditorialIdOrderByPrecioDesc(Long editorialId)</code>	<code>WHERE editorial_id = ? ORDER BY precio DESC</code>
<code>existsByTitulo(String titulo)</code>	igual que <code>findByTitulo</code> , pero devuelve boolean
<code>countByEditorialId(Long editorialId)</code>	igual, pero devuelve long — cuántas filas, no cuáles
<code>deleteByEditorialId(Long editorialId)</code>	<code>DELETE FROM libro WHERE editorial_id = ?</code> — borrado masivo

Estas piezas se combinan entre sí

Un nombre más largo, descompuesto:

```
List<Libro> findByTituloContainingIgnoreCaseOrderByPrecioAsc(String fragmento);
```

findBy (busca) + TitoloContainingIgnoreCase (el campo titulo contiene el fragmento, sin distinguir mayúsculas de minúsculas) + OrderByPrecioAsc (el resultado, ordenado por precio ascendente). Cada pieza se lee de izquierda a derecha, y cada una añade una condición o cláusula más a la consulta final.

Un nombre mal escrito no falla al llamarlo

⚠ Falla al arrancar la aplicación, no al usarlo

Spring Data JPA valida el nombre de cada método derivado contra los campos reales de la entidad al arrancar la aplicación, no la primera vez que lo llamas. Si te equivocas — por ejemplo, `findByTitle` en vez de `findByTitulo`, o `findByEditorial` cuando quieres filtrar por su id y falta el `Id` al final — la aplicación ni siquiera llega a arrancar: falla con un `PropertyReferenceException` que señala exactamente qué parte del nombre no ha sabido interpretar.

Es una de las pocas veces en que un error de "nombre mal escrito" se detecta antes de ejecutar nada, no en mitad de una petición real.

Ideas clave

- Un procedimiento almacenado es código con lógica que vive y se ejecuta dentro del gestor de base de datos, no en la aplicación.
- PostgreSQL usa PL/pgSQL como lenguaje procedural; un procedimiento se invoca con CALL, una función se puede usar dentro de un SELECT.
- Sus parámetros se pasan por posición y son de entrada (IN) por defecto — invertir el orden al invocarlo no da error, asigna mal los valores.
- Un procedimiento permite ejecutar una operación dentro de la propia base de datos, sin cargar y recorrer filas una a una desde Java.

Ideas clave (cont.)

- JdbcTemplate es el ayudante de Spring sobre JDBC puro — invoca procedimientos con parámetros seguros, sin la fontanería manual de abrir/cerrar recursos.
- Se inyecta como cualquier otro bean, junto a los repositorios de Spring Data JPA — ambos enfoques conviven en el mismo service.
- Las consultas derivadas por nombre generan SQL sin que escribas ninguna consulta — pero un nombre mal escrito falla al arrancar la aplicación, no al llamarlo.